



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo Práctico

21 de septiembre de 2026

Aprendizaje Automático I

Grupo 11

Integrante	LU	Correo electrónico
Bravo, Iván Agustín	001/01	email1@dominio.com
Kiss, María Delfina	72/23	kissdelfina@gmail.com
Lützow Holm, Eric	323/12	elholm90@gmail.com
Mendoza, Juan Cruz	004/01	email4@dominio.com
Rostan, Marcos Andrés	589/23	rostanmarcos@gmail.com



Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón Cero + Infinito)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Conmutador: (+54 11) 5285-9721 / 5285-7400

<https://dc.uba.ar>

1. Introducción

En este trabajo práctico buscamos generar un modelo para determinar si una muestra de células tiene pronóstico de crecimiento anómalo o descontrolado, conocido como hiperplasia. Para eso, contamos con un dataset de 500 instancias proveniente de pacientes con lesiones pretumorales, en el que se midió la expresión de 200 genes. Cada instancia fue etiquetada como verdadero o buen pronóstico si no hubo indicios de una nueva hiperplasia, o falso o mal pronóstico en caso contrario. Nuestra tarea fue probar diferentes algoritmos de aprendizaje automático con distintas configuraciones en busca del que diera mejores resultados en la clasificación. Los algoritmos que usamos fueron árboles de decisión, K vecinos más cercanos (KNN), support vector machines (SVM), análisis discriminante lineal (LDA) y Naive Bayes. Utilizamos el lenguaje Python. El notebook con el código completo se puede consultar aquí [En este informe mostramos solo el código mínimo para ilustrar los pasos más importantes.](#)

2. Separación de datos

Queremos partir el dataset de manera de tener una partición de held-out (conjunto de control o de evaluación) con la que evaluar, al final del trabajo, la configuración que elijamos. Para eso, podemos tomar el estándar de usar el 15 % del total. Luego, es muy importante que esa partición sea representativa del dataset: que la proporción de etiquetas sea aleatoria y balanceada. Es decir, que la proporción de True y False sea la misma que en el resto del dataset. Además, ese 15 % no debe tener una propiedad que el resto del dataset no tenga y viceversa. Por ejemplo, que los valores de ciertos atributos sean muy diferentes al del resto.

Sabemos que del total, el 72 % de las instancias corresponden a 0 (falsos o mal pronóstico) y 28 % a 1 (verdaderos o buen pronóstico). La propuesta final entonces es mezclar primero las instancias y luego tomar esas proporciones para particionar el held-out. Lo implementamos con los métodos `groupby` (agrupar por target) y luego `sample` (tomar una muestra al azar) de Pandas, fijando una semilla para que la partición sea reproducible. Queremos $0,15 \cdot 359$ de etiqueta 0 y $0,15 \cdot 141$ de etiqueta 1, o sea, 54 y 21 respectivamente, lo que deja un held-out de 75 instancias y un conjunto de desarrollo de 425.

Luego de esta separación, procedemos a construir nuestros modelos.

3. Construcción de modelos

Buscamos construir y evaluar modelos de árboles de decisión, obteniendo una estimación realista de su performance. Como primer paso separamos el conjunto de desarrollo en entrenamiento y validación de manera proporcional a sus etiquetas, usando `train_test_split` de `sklearn` estratificado, y nos quedamos con un 0,8 de entrenamiento y 0,2 de validación. Sobre esa partición entrenamos un árbol con altura máxima 3 y el resto de los hiperparámetros en sus valores por defecto.

De este árbol obtuvimos un accuracy de 0.6471 sobre el subconjunto de validación. Ese valor sale de una única partición de 85 instancias, por lo que depende bastante de cuáles hayan caído del lado de validación. Para obtener una estimación más estable repetimos la medición con validación cruzada sobre el conjunto de desarrollo completo, con un K-fold estratificado de 5 particiones de manera que cada fold conserve la proporción de etiquetas. Para cada fold calculamos Accuracy, Area Under the Precision-Recall Curve (AUPRC) y Area Under the Receiver Operating Characteristic Curve (AUCROC), y además el score global utilizando únicamente los folds de validación. Los resultados se pueden observar en la tabla 1.

Permutación	Acc (train)	Acc (val)	AUPRC (train)	AUPRC (val)	AUCROC (train)	AUCROC (val)
1	0.8265	0.6000	0.7153	0.3265	0.8626	0.5697
2	0.8265	0.7412	0.6862	0.4213	0.8037	0.6411
3	0.8353	0.6235	0.6981	0.2895	0.8506	0.4754
4	0.8265	0.6471	0.6900	0.3594	0.8429	0.6117
5	0.7971	0.6824	0.6491	0.3574	0.8538	0.5683
Promedios	0.8224	0.6588	0.6878	0.3508	0.8427	0.5732
Global	(NO)	0.6588	(NO)	0.3404	(NO)	0.5610

Tabla 1: Métricas de un árbol de altura máxima 3 con criterio Gini, por fold de la validación cruzada estratificada de 5 particiones sobre el conjunto de desarrollo. Global se calcula sobre las predicciones acumuladas de los 5 folds de validación

Para interpretar estos valores necesitamos un punto de comparación, así que evaluamos con la misma validación cruzada un clasificador que ignora los atributos y predice siempre la clase mayoritaria. Alcanza un accuracy de 0.7176, un AUPRC de 0.2824 que no es otra cosa que la proporción de positivos, y un AUCROC de 0.5000 porque asigna el mismo score a todas las instancias.

Luego exploramos 6 combinaciones de altura máxima y criterio de corte, manteniendo validación cruzada con $K = 5$ y generando las combinaciones con `ParameterGrid` de scikit-learn. Probamos alturas máximas de 3, 5 e infinita, con criterio de corte Gini y Entropía. Reutilizamos las mismas particiones para que los valores sean comparables, lo que se verifica en que la fila de altura 3 con Gini reproduce exactamente el resultado de la tabla 1. Los resultados se muestran en la tabla 2

Altura máxima	Criterio de corte	Accuracy (training)	Accuracy (validación)
3	Gini	0.8224	0.6588
5	Gini	0.9482	0.6447
Infinito	Gini	1.0000	0.6400
3	Entropía	0.8029	0.6541
5	Entropía	0.9253	0.6612
Infinito	Entropía	1.0000	0.5953

Tabla 2: Accuracy promedio en entrenamiento y validación de árboles de decisión, según altura máxima y criterio de corte, con la misma validación cruzada estratificada de 5 particiones sobre el conjunto de desarrollo.

De la tabla 1 vemos que el modelo base, con altura máxima 3 y criterio Gini, logró un accuracy de validación de 0.6588, por debajo del 0.7176 del clasificador trivial. Su AUCROC de validación es 0.5732 en promedio y 0.5610 global, apenas por encima del 0.5000 de referencia, y su AUPRC de 0.3508 mejora muy poco el 0.2824 que ya da predecir siempre la clase mayoritaria. El modelo base entonces casi no captura señal, aunque la brecha con entrenamiento es amplia, 0.8224 contra 0.6588 en accuracy y 0.8427 contra 0.5732 en AUCROC.

En la tabla 2 se observa el comportamiento típico del overfitting. Al relajar la altura máxima el accuracy de entrenamiento sube hasta 1.0000, es decir que el árbol memoriza las 340 instancias de cada fold, mientras que el de validación no mejora. Con Gini la caída es pareja, de 0.6588 a 0.6400, y con Entropía el peor caso es el árbol sin restricción, que baja a 0.5953.

Ninguna de las 6 configuraciones supera el 0.7176 del clasificador trivial, y el rango completo de la columna de validación, de 0.5953 a 0.6612, es más angosto que la variación entre folds de una misma configuración, que en la tabla 1 va de 0.6000 a 0.7412. Por eso no consideramos que ninguna combinación sea preferible a las demás. Tampoco el accuracy es la métrica más adecuada para este problema, porque con un 72 % de instancias negativas premia al modelo que ignora la clase minoritaria, que es justamente la que interesa detectar, y por eso de acá en adelante usamos AUCROC. En conjunto, las dos tablas muestran que un árbol de decisión no alcanza para estos 200 atributos, lo que motiva explorar otras familias de algoritmos.

4. Comparación de algoritmos

En esta sección probamos árboles de decisión, KNN y SVM, con varias combinaciones de hiperparámetros. Usamos la AUCROC como métrica de comparación.

4.1. Árboles de decisión

Como hiperparámetros tuvimos en cuenta la profundidad máxima, la mínima cantidad de instancias que necesita un nodo para dividirse, la mínima cantidad de instancias de cada hoja, el criterio (Gini o Entropía) y la máxima cantidad de atributos, probando la raíz cuadrada y el logaritmo de la cantidad total.

```
parametros_arbol = {"max_depth": [3, 5, 10, 20, None],
                    "min_samples_split": [2, 5, 10, 20],
                    "min_samples_leaf": [1, 5, 10, 15],
                    "criterion": ["gini", "entropy"],
                    "max_features": ["sqrt", "log2", None]}

arbol_base = DecisionTreeClassifier(random_state=random_state)

buscador_arbol = RandomizedSearchCV(estimator=arbol_base, param_distributions=parametros_arbol,
                                   n_iter=50, scoring="roc_auc", cv=skf, random_state=random_state)

buscador_arbol.fit(X_dev, y_dev)
```

Los mejores hiperparámetros encontrados fueron: 'min_samples_split': 10, 'min_samples_leaf': 15, 'max_features': 'log2', 'max_depth': 5, 'criterion': 'gini' Con un AUCROC score de 0.6740.

4.2. KNN

Para KNN, primero hay que tener en cuenta que suele ser mejor escalar las features. Como pueden tener escalas muy distintas, las features de mayor rango pueden llegar a dominar la distancia. Hiperparámetros:

- **n_neighbours**: mientras un k chico se puede llegar a ajustar al ruido local de cada datapoint, un k grande simplemente se aproxima a predecir la clase mayoritaria.
- **weights**: permite que los puntos más cercanos pesen más.
- **metric**: que es la distancia que usamos para medir cercanía.

```
parametros_knn = {"knn__n_neighbors": [3, 5, 7, 10, 15, 21, 31, 41, 51, 61, 75],
                  "knn__weights": ["uniform", "distance"],
                  "knn__metric": ["euclidean", "manhattan"],
                  "scaler": [StandardScaler(), None]}

knn_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier())])

buscador_knn = RandomizedSearchCV(estimator=knn_pipe, param_distributions=parametros_knn,
                                   n_iter=50, scoring="roc_auc", cv=skf, random_state=random_state)

buscador_knn.fit(X_dev, y_dev)
```

Mejores hiperparámetros encontrados para KNN: 'scaler': None, 'knn_weights': 'distance', 'knn_n_neighbors': 21, 'knn_metric': 'manhattan' El mejor score AUCROC fue 0.8474.

4.3. SVM

Para SVM, también necesitamos escalar los features. Hiperparámetros:

- **c**: controla la regularización del SVM. Cuanto mayor es, obliga a SVM a tener un margen estricto, lo que puede llevar a overfitting, mientras que cuanto más chico es, tolera más puntos mal clasificados.
- **kernel**: nos dice si la frontera es lineal en el espacio de features, o se hace un mapping mediante el kernel trick para hacer un hiperplano en otro espacio.
- **degree**: para kernels polinómicos solamente debemos decidir hasta qué grado puede ser el polinomio.
- **gamma**: para el kernel RBF y poly, gamma define el radio de influencia de cada support vector.

Para Gamma y c, se suelen probar diferentes órdenes de magnitud. Para degree, grados mayores a 3 puede generar un espacio de features enorme y sobreajustar con pocas instancias.

```
parametros_svm = [  
    {"svm__kernel": ["linear"], "svm__C": [0.01, 0.1, 1, 10, 100, 1000], "scaler": [StandardScaler(),  
    None]},  
    {"svm__kernel": ["rbf"], "svm__C": [0.01, 0.1, 1, 10, 100, 1000],  
    "svm__gamma": ["scale", 0.0001, 0.001, 0.01, 0.1, 1], "scaler": [StandardScaler(), None]},  
    {"svm__kernel": ["poly"], "svm__C": [0.01, 0.1, 1, 10, 100, 1000],  
    "svm__degree": [2, 3], "svm__gamma": ["scale", 0.0001, 0.001, 0.01, 0.1, 1], "scaler": [  
    StandardScaler(), None]}]  
  
svm_pipe = Pipeline([  
    ("scaler", StandardScaler()),  
    ("svm", SVC(random_state=random_state))])  
  
buscador_svm = RandomizedSearchCV(estimator=svm_pipe, param_distributions=parametros_svm,  
    n_iter=50, scoring="roc_auc", cv=skf, random_state=random_state)  
  
buscador_svm.fit(X_dev, y_dev)
```

Los mejores hiperparámetros encontrados para SVM fueron 'svm_kernel': 'poly', 'svm_gamma': 1, 'svm_degree': 2, 'svm_C': 1000, 'scaler': None El mejor score AUCROC fue 0.8587.

Vemos que las top 5 mejores configuraciones tienen la misma AUCROC y sólo cambian el C y Gamma, indicando que sin escalar los mismos no tienen relevancia.

Índice	Params	Mean Test Score	Std Test Score
2	{'svm__kernel': 'poly', 'svm__gamma': 1, ...}	0.858743	0.055193
9	{'svm__kernel': 'poly', 'svm__gamma': 0.0001, ...}	0.858743	0.055193
23	{'svm__kernel': 'poly', 'svm__gamma': 1, ...}	0.858743	0.055193
29	{'svm__kernel': 'poly', 'svm__gamma': 0.01, ...}	0.858743	0.055193
25	{'svm__kernel': 'poly', 'svm__gamma': 0.01, ...}	0.858743	0.055193

Tabla 3: Mejores configuraciones obtenidas mediante búsqueda de hiperparámetros.

Resulta muy llamativo ver que tanto para SVM y KNN, las mejores configuraciones de hiperparámetros según AUCROC fueron aquellas que no utilizaron el Scaler. Esto indica que las features con un rango más grande terminaron siendo aquellas que también tenían mayor poder predictivo.

4.4. LDA y Naive Bayes

Por último, añadimos a la comparación LDA y Naive Bayes.

```
lda = LinearDiscriminantAnalysis()
scores_lda = cross_val_score(lda, X_dev, y_dev, cv=skf, scoring="roc_auc")

nb = GaussianNB()
scores_nb = cross_val_score(nb, X_dev, y_dev, cv=skf, scoring="roc_auc")
```

Fold	LDA	Naive Bayes
1	0.7131	0.7425
2	0.6919	0.7582
3	0.6291	0.7589
4	0.6086	0.6926
5	0.5608	0.7357
Promedio	0.6407	0.7376

Tabla 4: AUCROC por fold y promedio para LDA y Naive Bayes.

Vemos que el AUCROC de LDA es el peor con $AUC = 0.6407$. Esto puede deberse a que se está estimando una matriz de covarianza de 200×200 (por las 200 features con que se entrenan) con pocas instancias de train por fold, lo que resulta en una estimación ruidosa y casi singular. En este caso, podríamos utilizar el hiperparámetro de `shrinkage` que regulariza la matriz de covarianza.

Por otro lado, Naive Bayes, incluso sin haber hecho una búsqueda de hiperparámetros, tuvo un $AUC = 0.7376$, similar a la performance de SVM y KNN. Este modelo, al asumir independencia entre los features, sólo estima la media y varianza por cada gen, por lo que cada una de las estimaciones puede ser buena incluso con pocos datos. Uno de los hiperparámetros que podríamos modificar es `var_smoothing`, el cual añade una fracción de la varianza más grande de las features a cada una de las estimaciones de varianza. Esto funciona como una regularización de la varianza, y también ayuda a que si hay una feature con varianza casi cero, un valor distinto en test no reciba probabilidad casi cero, dominando el producto.

5. Evaluación de performance

6. Conclusiones

7. Discusión

8. Bibliografía consultada